



中山大學
SUN YAT-SEN UNIVERSITY

运算符重载 (下)

中山大学计算机学院



主讲人：郑贵锋

zhenggf@mail.sysu.edu.cn



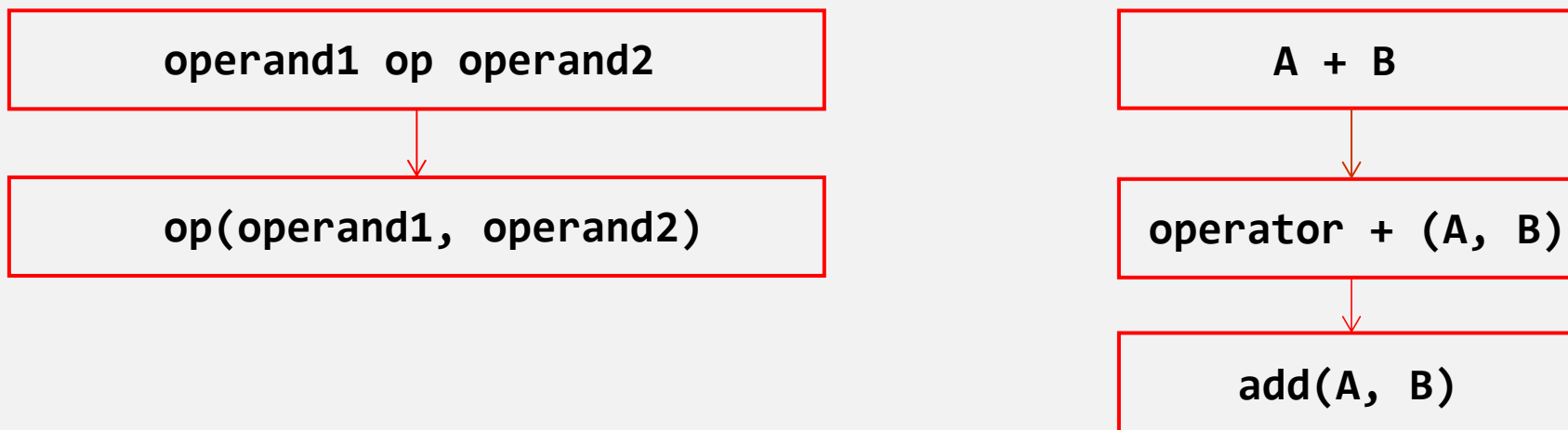
中山大学MOOC课程组



运算符重载 (复习)

C++为了增强代码的可读性引入了运算符重载，运算符重载是具有特殊函数名的函数，也具有返回值类型，函数名和参数列表。

重载的运算符可以理解为带有特殊名称的函数，函数名是由关键字 `operator` 和其后要重载的运算符符号构成的。



运算符重载的方法（复习）

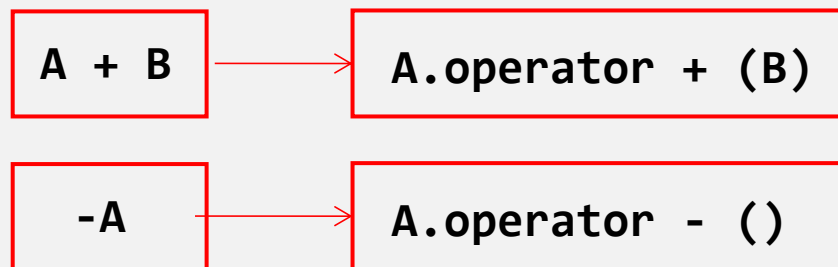
运算符重载的两种方法：

- 类成员运算符重载

- `return_type class_name::operator op(operand2) {}`
- 重载二元运算符时，成员运算符函数只需显式传递一个参数（即二元运算符的右操作数），而左操作数则是该类对象本身，通过 `this` 指针隐式传递。
- 重载一元运算符时，成员运算符函数没有参数，操作数是该类对象本身，通过 `this` 指针隐式传递

- 友元运算符重载

- `return_type operator op(operand1, operand2) {}`
- 后面会详细说明





注意事项（一）：自增运算符（复习）

```
class Integer {  
    int x;  
public:  
    Integer(int x=0):x(x)    {  
    }  
    Integer& operator ++ () {  
        cerr<<"prefix is invoked"<<endl;  
        ++x;  
        return *this;  
    }  
    Integer operator ++ (int) {  
        cerr<<"suffix is invoked"<<endl;  
        return Integer(x++);  
    }  
    void print()    {  
        cout<<x<<endl;  
    }  
};
```

```
int main() {  
    Integer a=3;  
    a.print();  
    Integer b=++a;  
    a.print();  
    b.print();  
    Integer c=a++;  
    a.print();  
    c.print();  
    return 0;  
}
```

输出结果：

3
prefix is invoked
4
4
suffix is invoked
5
4

注意事项（二）：赋值运算符（复习）

自定义的类会默认有一个缺省的赋值运算符，每个成员变量会直接拷贝值：

```
class Integer {  
    int x;  
public:  
    Integer(int x=0):x(x)    {  
    }  
    void print()    {  
        cout<<x<<endl;  
    }  
};  
  
int main() {  
    Integer a=3, b;  
    a.print();  
    b=a; // default operator =  
    b.print();  
    return 0;  
}
```

输出结果：

3
3

注意事项（二）：赋值运算符（复习）

赋值运算符（`operator=`）有特殊性质：细节见[复制赋值与移动赋值](#)。

对规范的复制赋值运算符，期待其在[自赋值时不进行操作](#) ，并按引用返回 lhs：

```
// 假设对象保有可重用存储，例如一个分配于堆的缓冲区 mArray
T& operator=(const T& other) // 复制赋值
{
    if (this != &other) { // 期待检查自赋值
        if (other.size != size) { // 存储不可复用
            delete[] mArray;      // 销毁 this 中的存储
            size = 0;
            mArray = nullptr;     // 在下一行抛出的情况下维持不变式
            mArray = new int[other.size]; // 分配 this 中的存储
            size = other.size;
        }
        std::copy(other.mArray, other.mArray + other.size, mArray);
    }
    return *this;
}
```



注意事项（三）：移位运算符（复习）

在我们初学C++的时候，可能会有这样一个疑问：

- 明明 <<, >> 是数字的移位运算符，为什么他却可以用来读入cin>>x和输出cout<<x呢

现在我们知道了，其实这个就是运算符<<和>>的重载。

同理，我们也可以对自己定义的类做重载：

```
struct Integer {  
    int x;  
};  
  
istream& operator >> (istream& is, Integer& Int) {  
    is>>Int.x;  
    return is;  
}  
  
ostream& operator << (ostream& os, const Integer& Int) {  
    os<<Int.x;  
    return os;  
}  
  
int main() {  
    Integer x;  
    cin>>x;  
    cout<<x<<endl;  
    return 0;  
}
```



注意事项（四）：函数对象

当定义了operator ()的类对象调用此操作符时，其表现形式如同普通函数调用一般，因此取名叫函数对象。举个最简单的例子：

```
class cmp {  
public:  
    bool operator() (const int& a, const int& b) {  
        return a<b;  
    }  
};  
  
int main() {  
    cmp f;  
    cout<<f(1,2)<<endl;  
    cout<<f(2,1)<<endl;  
    return 0;  
}
```

输出结果：

1
0

注意事项（四）：函数对象

但函数对象实际上也是个对象，所以可以拥有自己的成员变量，从而执行带谓词的函数执行：

```
class GreaterThan {  
    int baseline;  
public:  
    GreaterThan(int x):baseline(x) {  
    }  
    bool operator() (const int& x) {  
        return x>baseline;  
    }  
};  
  
int main() {  
    GreaterThan G1(10), G2(20);  
    cout<<G1(15)<<endl;  
    cout<<G2(15)<<endl;  
    return 0;  
}
```

输出结果：

1
0



注意事项（四）：函数对象

所以，函数对象相比普通的函数有一个非常重要的用途，即作为谓词函数（Predicate）。

谓词函数通常用来对传进来的参数进行判断，并返回布尔值。标准库中有大量的函数都定义了多个重载版本，其中包含由用户提供谓词函数的，比如：find_if, remove_if, 等等。

现在假设我们有一串数字，要从中找出第一个大于10的数字以及第一个大于20的数字：

```
class GreaterThan {
    int baseline;
public:
    GreaterThan(int x):baseline(x) {
    }
    bool operator() (const int& x) {
        return x>baseline;
    }
};

int main() {
    vector<int> a={5,10,15,20,25};
    // find_if return a iterator
    cout<<*find_if(a.begin(), a.end(), GreaterThan(10))<<endl;
    cout<<*find_if(a.begin(), a.end(), GreaterThan(20))<<endl;
    return 0;
}
```

```
bool GreaterThan10(const int& x) {
    return x>10;
}

bool GreaterThan20(const int& x) {
    return x>20;
}

int main() {
    vector<int> a={5,10,15,20,25};
    // find_if return a iterator
    cout<<*find_if(a.begin(), a.end(), GreaterThan10)<<endl;
    cout<<*find_if(a.begin(), a.end(), GreaterThan20)<<endl;
    return 0;
}
```

使用函数对象使得代码更具有可读性！



问题引入

读入输出能重载当然是一件方便的事，但有时候类的成员是 `private` 的，外部函数无法访问，这又该如何解决？

```
class Integer {  
    int x;  
};  
  
istream& operator >> (istream& is, Integer& Int) {  
    is>>Int.x; // error: private, thus unavailable  
    return is;  
}  
  
ostream& operator << (ostream& os, const Integer& Int)  
    os<<Int.x; // error: private, thus unavailable  
    return os;  
}
```

Integer_type_list);

的函数放进类内部，并在前面加上 `friend` 关系，那么有变量以及私有函数。



友元函数

答案就是友元函数 (friend function) 。

- `friend return_type function_name(parameter_type_list);`

顾名思义，友元函数就是给“朋友”看的函数，将正常声明的函数放进类内部，并在前面加上 `friend` 关键字，那么这个函数虽然不属于类，但却可以访问类的私有变量以及私有函数。

```
class Integer {  
    int x;  
    friend istream& operator >> (istream& is, Integer& Int);  
    friend ostream& operator << (ostream& os, const Integer& Int);  
};  
  
istream& operator >> (istream& is, Integer& Int) {  
    is>>Int.x; // private but friend, unavailable  
    return is;  
}  
  
ostream& operator << (ostream& os, const Integer& Int) {  
    os<<Int.x; // private but friend thus unavailable  
    return os;  
}
```



友元函数

类的成员函数也是函数的一种，所以其他类的成员函数也可以是友元函数！

```
class Integer;
struct Cmp {
    bool operator () (const Integer& a, const Integer& b);
};

class Integer {
    int x;
public:
    Integer(int x=0):x(x) {
    }
    friend bool Cmp::operator () (const Integer& a, const Integer& b);
};

bool Cmp::operator () (const Integer& a, const Integer& b) {
    return a.x<b.x;
}
```

友元类

但有些时候，其他类的成员函数可能会很多，一个一个的声明为友元函数会比较麻烦。

所以我们就可以直接声明友元类：

- 一个类 A 可以将另一个类 B 声明为自己的友元，那么类 B 的所有成员函数就都可以访问类 A 对象的私有成员
- `friend class B;` （在类 A 的内部）

```
class Integer;
struct Cmp {
    bool operator () (const Integer& a, const Integer& b);
};

class Integer {
    int x;
public:
    Integer(int x=0):x(x) {
    }
    friend Cmp;
};

bool Cmp::operator () (const Integer& a, const Integer& b) {
    return a.x<b.x;
}
```


操作符重载 函数成员 or 友元函数?

1 必须是成员函数;

a=b	(a).operator= (b)	不能是非成员	给定 <code>std::string s;</code> , <code>s = "abc";</code> 调用 <code>s.operator= ("abc")</code>
a(b...)	(a).operator() (b...)	不能是非成员	给定 <code>std::random_device r;</code> , <code>auto n = r();</code> 调用 <code>r.operator() ()</code>
a[b]	(a).operator[] (b)	不能是非成员	给定 <code>std::map<int, int> m;</code> , <code>m[1] = 2;</code> 调用 <code>m.operator[] (1)</code>
a->	(a).operator-> ()	不能是非成员	给定 <code>auto p = std::make_unique<S>();</code> <code>p->bar();</code> 调用 <code>p.operator->()</code>

2 必须是类外定义运算符 (友元函数) ;

?



问题引入

在学习了这节内容后，有同学可能会写出类似于下面的代码：

- Integer 可以利用字符串进行构造，并且也有一个名字为compare的函数对象
- 但是ta之后又写了其他代码，写着写着他已经忘记了compare的定义...
- 而这时候他需要比较两个字符串的大小；哎，有个函数对象compare，他直接就调用了这个compare：

```
class Integer;
struct Cmp {
    bool operator () (const Integer& a, const Integer& b);
};

class Integer {
    int x;
public:
    // atoi is a stl function that convert a c-string to int
    Integer(const char* s):x(atoi(s)) {
    }
    Integer(int x=0):x(x) {
    }
    friend Cmp;
};

bool Cmp::operator () (const Integer& a, const Integer& b) {
    return a.x<b.x;
}
```

```
int main() {
    Integer a="3", b="4";
    Cmp compare;
    cout<<compare(a,b)<<endl;
    // some codes
    cout<<compare("a","b")<<endl; // unexpexcted behavior
    return 0;
}
```



对象隐式转换

如果对象 T 存在构造函数 $T(T1)$, 则 $T1$ 类型对象 (实参) 可隐式转为 T 类型对象 (形参)

```
/* 隐式转换 */
```

```
...
```

```
class Integer {
```

```
    int x;
```

```
public:
```

```
    Integer(int x=0):x(x) {}
```

```
    friend Integer operator+(const Integer &lhs,const Integer &rhs){
```

```
        return lhs.x + rhs.x;
```

```
    }
```

```
    friend ostream& operator<<(ostream& o, const Integer &hs) {
```

```
        o << hs.x;
```

```
        return o;
```

```
    }
```

```
};
```

```
int main() {  
    string s;  
    s = "Hello";  
    cout << s << endl;  
  
    Integer i1(3), i2;  
    i2 = 1.1 + i1;  
    cout << i2 << endl;  
}
```

请指出程序中的隐式转换?



对象隐式转换

如果对象 T 存在构造函数 $T(T1)$, 则 $T1$ 类型对象 (实参) 可隐式转为 T 类型对象 (形参)

```
/* 隐式转换 */
```

```
...
```

```
class Integer {
```

```
    int x;
```

```
public:
```

```
    Integer(int x=0):x(x) {}
```

```
    friend Integer operator+(const Integer &lhs,const Integer &rhs){
```

```
        return lhs.x + rhs.x; ① int -> Integer
```

```
    }
```

```
    friend ostream& operator<<(ostream& o, const Integer &hs) {
```

```
        o << hs.x;
```

```
        return o;
```

```
    }
```

```
};
```

```
int main() {
```

```
    string s; ②
```

```
    s = "Hello"; const char* -> string
```

```
    cout << s << endl;
```

```
    Integer i1(3), i2; ③
```

```
    i2 = 1.1 + i1; double -> int
```

```
    cout << i2 << endl;
```

```
}
```

请指出程序中的隐式转换?



隐式转换-重载协议-const

如果重载的函数参数一样，可以通过转换到某个重载函数，编译会如何哪个版本的选择？

- (1) 类型直接匹配的优先选择；
- (2) `const` 类型实参匹配 `const` 版本
- (3) 非 `const` 实参优先匹配非 `const` 版本。

没有则隐式转换为 `const` 版本匹配

程序运行结果：

no_const 1,3,const 2

观察与练习：

1. 注释去除非 `const` 版本，编译运行
2. 注释去除 `const` 版本，编译

```
/* 隐式转换-重载协议-const */  
...  
class Integer {  
    int x;  
public:  
    Integer(int x=0):x(x) {}  
    friend ostream& operator<<(ostream& o, const Integer &hs) {  
        o << "const " << hs.x;  
        return o;  
    }  
    friend ostream& operator<<(ostream& o, Integer &hs) {  
        o << "no_const " << hs.x;  
        return o;  
    }  
};  
  
int main() {  
    Integer i1(1);  
    const Integer i2(2);  
    cout << i1 << "," << 3 << "," << i2 << endl;  
}
```



隐式转换-nullptr

C 语言零值常数有很多表示，如 0，NULL，
'\0'，C++ 右引入了 nullptr 表示空值指针
字面量。

右边的例子解释了 nullptr 的必要性

```
/* 隐式转换-nullptr */
...
class Integer {
    int x;
public:
    Integer(int x=0):x(x) {}
    friend Integer operator+(const Integer &lhs, Integer rhs){
        return lhs.x + rhs.x;
    }
    friend Integer operator+(const Integer &lhs, Integer *rhs){
        if (rhs) return lhs.x + rhs->x + 2000;
        else return lhs.x + 1000;
    }
    friend ostream& operator<<(ostream& o, const Integer &hs) {
        o << hs.x; return o;
    }
};

class Girlfriend { /* ... */};
void kissGirlfriend(Girlfriend* gf) {cout << "pointer"<<endl;};
void kissGirlfriend(int gfID){cout << "int"<<endl;};

int main() {
    kissGirlfriend(nullptr); // 指针类型字面量 (void *) 0
    kissGirlfriend(0); // 尝试 NULL 取代 0

    Integer i1(1), i2;
    i2 = i1 + 0; // 尝试 1,nullptr,NULL,&i1 取代 0
    cout << i2 << endl;
}
```




explicit

C++ 关键字 `explicit`，用于关闭这种自动类型转化的特性。

即被 `explicit` 关键字修饰的类构造函数，不能进行自动地隐式类型转换，只能显式地进行类型转换。

```
class Integer;
struct Cmp {
    bool operator () (const Integer& a, const Integer& b);
};

class Integer {
    int x;
public:
    // atoi is a stl function that convert a c-string to int
    explicit Integer(const char* s):x(atoi(s)) {
    }
    Integer(int x=0):x(x) {
    }
    friend Cmp;
};

bool Cmp::operator () (const Integer& a, const Integer& b) {
    return a.x<b.x;
}
```

```
int main() {
    Integer a("3"), b("4");
    Cmp compare;
    cout<<compare(a,b)<<endl; // of course ok
    // some codes
    cout<<compare(Integer("1"),Integer("2"))<<endl; // explicit construction, ok
    cout<<compare((Integer)"1",(Integer)"2")<<endl; // explicit conversion, ok
    cout<<compare("1","2")<<endl; // error
    cout<<compare("a","b")<<endl; // error
    return 0;
}
```



中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院



主讲人：郑贵锋

zhenggf@mail.sysu.edu.cn



中山大学MOOC课程组